

Staking Contract

Caldera

HALBORN

Staking Contract - Caldera

Prepared by:  HALBORN

Last Updated 03/28/2025

Date of Engagement: March 12th, 2025 - March 18th, 2025

Summary

100% ⓘ OF ALL REPORTED FINDINGS HAVE BEEN ADDRESSED

ALL FINDINGS	CRITICAL	HIGH	MEDIUM	LOW	INFORMATIONAL
16	0	0	2	2	12

TABLE OF CONTENTS

1. Introduction	
2. Assessment summary	
3. Test approach and methodology	
4. Caveats	
5. Static analysis report	
5.1 Description	
5.2 Output	
6. Risk methodology	
7. Scope	
8. Assessment summary & findings overview	
9. Findings & Tech Details	
9.1 Dos vulnerability when minimum stake amount is increased	
9.2 Reward rate not updated after change in rewards end timestamp	
9.3 Inconsistent upgradeable inheritance pattern	
9.4 Reward rate spike when approaching reward period end	
9.5 Centralization risk in manual reward rate controls	
9.6 Missing setter function for automaticstakingpercent	
9.7 Inconsistent input validation across contract parameters	

9.8 Ordering for nonreentrant modifier

9.9 Lack of upgradeable storage management pattern

9.10 Use of magic numbers

9.11 Lack of named mappings

9.12 Unoptimized for loops

9.13 Single step ownership transfer process

9.14 Redundant/inefficient code

9.15 Inconsistent function naming convention

9.16 Integer division in reward rate calculation could lead to zero rewards

1. Introduction

Caldera engaged Halborn to conduct a security assessment on their smart contracts beginning on March 12th, 2025 and ending on March 18th, 2025. The security assessment was scoped to the smart contracts provided to Halborn. Commit hashes and further details can be found in the Scope section of this report.

The Caldera codebase in scope consists of an airdrop contract for distributing tokens and a staking contract with dynamic-rate rewards, configurable stake lockup and unstake/withdrawal cooldown periods.

2. Assessment Summary

Halborn was provided 5 days for the engagement and assigned 1 full-time security engineer to review the security of the smart contracts in scope. The engineer is a blockchain and smart contract security expert with advanced penetration testing and smart contract hacking skills, and deep knowledge of multiple blockchain protocols.

The purpose of the assessment is to:

- Identify potential security issues within the smart contracts.
- Ensure that smart contract functionality operates as intended.

In summary, Halborn identified some improvements to reduce the likelihood and impact of risks, which were mostly addressed by the Caldera team. The main ones were the following:

- Allow caller to specify a call to the `syncRewardRate()` function after calling `setRewardsEndTimestamp()`.
- Allow users to call `withdrawAll()` and `stakeAll()` from the contract.
- Replace the import and inheritance of `ReentrancyGuard` with `ReentrancyGuardUpgradeable`.

3. Test Approach And Methodology

Halborn performed a combination of manual review of the code and automated security testing to balance efficiency, timeliness, practicality, and accuracy in regard to the scope of this assessment. While manual testing is recommended to uncover flaws in logic, process, and implementation; automated testing techniques help enhance coverage of smart contracts and can quickly identify items that do not follow security best practices.

The following phases and associated tools were used throughout the term of the assessment:

- Research into architecture, purpose and use of the platform.
- Smart contract manual code review and walkthrough to identify any logic issue.
- Thorough assessment of safety and usage of critical Solidity variables and functions in scope that could lead to arithmetic related vulnerabilities.
- Local testing with custom scripts (Foundry).
- Fork testing against main networks (Foundry).
- Static analysis of security for scoped contract, and imported functions (Slither).

4. Caveats

The current security assessment was limited to the following:

- The CalderaStaking contracts were reviewed in its entirety.
- The CalderaAirdrop contract assessment was scoped to changes provided in the following PR: <https://github.com/nate-caldera/era-contracts-halborn/pull/1/files#diff-d2e56e3589a19802e465115660928b576f44897d7443c4e0d3cfcfb0e895426a>.

It's important to note that despite these caveats, the security assessment aimed to provide a thorough evaluation of the protocol's security posture. However, the limitations mentioned above should be considered when interpreting the findings and recommendations.

5. Static Analysis Report

5.1 Description

Halborn used automated testing techniques to enhance the coverage of certain areas of the smart contracts in scope. Among the tools used was Slither, a Solidity static analysis framework. After **Halborn** verified the smart contracts in the repository and was able to compile them correctly into their abis and binary format, Slither was run against the contracts. This tool can statically verify mathematical relationships between Solidity variables to detect invalid or inconsistent usage of the contracts' APIs across the entire code-base.

The security team assessed all findings identified by the Slither software, however, findings with related to external dependencies are not included in the below results for the sake of report readability.

5.2 Output

The findings obtained as a result of the Slither scan were reviewed, and many were not included in the report because they were determined as false positives.

```
INFO:Detectors:
CalderaAirdrop._addressClaim(bytes32[],uint256) (src/CalderaAirdrop.sol#374-396) uses arbitrary from in transferFrom: IERC20(token).safeTransferFrom(airdropVault,sender,amount - automaticStakingAmount) (src/CalderaAirdrop.sol#395)
CalderaAirdrop._addressClaim(bytes32[],uint256) (src/CalderaAirdrop.sol#374-396) uses arbitrary from in transferFrom: IERC20(token).safeTransferFrom(airdropVault,address(this),automaticStakingAmount) (src/CalderaAirdrop.sol#392)
CalderaAirdrop._githubClaim(bytes32[],string,uint256) (src/CalderaAirdrop.sol#401-423) uses arbitrary from in transferFrom: IERC20(token).safeTransferFrom(airdropVault,sender,amount - automaticStakingAmount) (src/CalderaAirdrop.sol#422)
CalderaAirdrop._githubClaim(bytes32[],string,uint256) (src/CalderaAirdrop.sol#401-423) uses arbitrary from in transferFrom: IERC20(token).safeTransferFrom(airdropVault,address(this),automaticStakingAmount) (src/CalderaAirdrop.sol#419)
CalderaStaking.getReward() (src/CalderaStaking.sol#466-486) uses arbitrary from in transferFrom: IERC20(token).safeTransferFrom(rewardsVault,msg.sender,claimableRewardAmount) (src/CalderaStaking.sol#484)
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#arbitrary-from-in-transferfrom
INFO:Detectors:
CalderaStaking.stakes (src/CalderaStaking.sol#74) is never initialized. It is used in:
- CalderaStaking.stake(uint256) (src/CalderaStaking.sol#322-346)
- CalderaStaking.stakeFor(address,uint256) (src/CalderaStaking.sol#355-376)
- CalderaStaking.unstake(uint256) (src/CalderaStaking.sol#381-424)
- CalderaStaking.getReward() (src/CalderaStaking.sol#466-486)
- CalderaStaking.getUserClaimableRewardAmount(address) (src/CalderaStaking.sol#507-519)
- CalderaStaking.getUserTotalRewardAmount(address) (src/CalderaStaking.sol#525-538)
- CalderaStaking.getUserTotalStakeAmount(address) (src/CalderaStaking.sol#542-547)
- CalderaStaking.getUserStakeCount(address) (src/CalderaStaking.sol#552-554)
- CalderaStaking._cleanUserStakes(address) (src/CalderaStaking.sol#598-632)
- CalderaStaking._getUserUnlockedStakeAmount(address) (src/CalderaStaking.sol#669-678)
CalderaStaking.withdrawalRequests (src/CalderaStaking.sol#75) is never initialized. It is used in:
- CalderaStaking.unstake(uint256) (src/CalderaStaking.sol#381-424)
- CalderaStaking.withdraw(uint256) (src/CalderaStaking.sol#428-462)
- CalderaStaking.getUserTotalWithdrawalRequestsAmount(address) (src/CalderaStaking.sol#559-564)
- CalderaStaking.getUserWithdrawalRequestCount(address) (src/CalderaStaking.sol#569-571)
- CalderaStaking._cleanUserWithdrawalRequests(address) (src/CalderaStaking.sol#636-653)
- CalderaStaking._getUserWithdrawableAmount(address) (src/CalderaStaking.sol#683-691)
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#uninitialized-state-variables
INFO:Detectors:
CalderaAirdrop.initialize(CalderaAirdrop.InitializeParams) (src/CalderaAirdrop.sol#153-285) ignores return value by IERC20(token).approve(params.staking,type()(uint256).max) (src/CalderaAirdrop.sol#284)
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#unused-return
INFO:Detectors:
CalderaAirdrop.setAirdropVault(address).airdropVault_ (src/CalderaAirdrop.sol#219) lacks a zero-check on :
- airdropVault = airdropVault_ (src/CalderaAirdrop.sol#228)
CalderaAirdrop.setGithubSigner(address).githubSigner_ (src/CalderaAirdrop.sol#226) lacks a zero-check on :
- githubSigner = githubSigner_ (src/CalderaAirdrop.sol#227)
CalderaStaking.initialize(address,address,address,address,uint256,uint256,uint256,uint256).stakeForSenderAddress_ (src/CalderaStaking.sol#158) lacks a zero-check on :
- stakeForSenderAddress = stakeForSenderAddress_ (src/CalderaStaking.sol#178)
CalderaStaking.setStakeForSenderAddress(address).stakeForSenderAddress_ (src/CalderaStaking.sol#222) lacks a zero-check on :
- stakeForSenderAddress = stakeForSenderAddress_ (src/CalderaStaking.sol#224)
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#missing-zero-address-validation
INFO:Detectors:
Reentrancy in CalderaAirdrop.combinedClaim(uint256,bytes32[],bytes,string,uint256,bytes32[],bytes) (src/CalderaAirdrop.sol#328-346):
  External calls:
  - _addressClaim(addressProof,addressAmount) (src/CalderaAirdrop.sol#344)
    - returndata = address(token).functionCall(data) (lib/ozopenzeppelin-contracts/contracts/token/ERC20/utils/SafeERC20.sol#96)
    - (success, returndata) = target.call{value: value}(data) (lib/ozopenzeppelin-contracts/contracts/utils/Address.sol#87)
    - IERC20(token).safeTransferFrom(airdropVault,address(this),automaticStakingAmount) (src/CalderaAirdrop.sol#392)
    - CalderaStaking(staking).stakeFor(sender,automaticStakingAmount) (src/CalderaAirdrop.sol#393)
    - IERC20(token).safeTransferFrom(airdropVault,sender,amount - automaticStakingAmount) (src/CalderaAirdrop.sol#395)
  - _githubClaim(githubProof,githubUsername,githubAmount) (src/CalderaAirdrop.sol#345)
    - returndata = address(token).functionCall(data) (lib/ozopenzeppelin-contracts/contracts/token/ERC20/utils/SafeERC20.sol#96)
    - (success, returndata) = target.call{value: value}(data) (lib/ozopenzeppelin-contracts/contracts/utils/Address.sol#87)
    - IERC20(token).safeTransferFrom(airdropVault,address(this),automaticStakingAmount) (src/CalderaAirdrop.sol#419)
    - CalderaStaking(staking).stakeFor(sender,automaticStakingAmount) (src/CalderaAirdrop.sol#420)
    - IERC20(token).safeTransferFrom(airdropVault,sender,amount - automaticStakingAmount) (src/CalderaAirdrop.sol#422)
  External calls sending eth:
  - _addressClaim(addressProof,addressAmount) (src/CalderaAirdrop.sol#344)
```

```

- (success, returndata) = target.call{value: value}(data) (lib/openzeppelin-contracts/contracts/utils/Address.sol#87)
- _githubClaim(githubProof, githubUsername, githubAmount) (src/CalderaAirdrop.sol#345)
- (success, returndata) = target.call{value: value}(data) (lib/openzeppelin-contracts/contracts/utils/Address.sol#87)
Event emitted after the call(s):
- GithubClaimed(sender, githubUsername, amount) (src/CalderaAirdrop.sol#415)
- _githubClaim(githubProof, githubUsername, githubAmount) (src/CalderaAirdrop.sol#345)
Reference: https://github.com/crytic/sliether/wiki/Detector-Documentation#reentrancy-vulnerabilities-3
INFO:Detectors:
CalderaAirdrop._requireWithinClaimPeriod() (src/CalderaAirdrop.sol#426-433) uses timestamp for comparisons
Dangerous comparisons:
- block.timestamp < startTime (src/CalderaAirdrop.sol#427)
- block.timestamp > endTime (src/CalderaAirdrop.sol#430)
CalderaStaking.setRewardsEndTimeStamp(uint256) (src/CalderaStaking.sol#263-268) uses timestamp for comparisons
Dangerous comparisons:
- rewardsEndTimeStamp == 0 || rewardsEndTimeStamp_ < block.timestamp (src/CalderaStaking.sol#264)
CalderaStaking.syncRewardRate(uint256) (src/CalderaStaking.sol#284-301) uses timestamp for comparisons
Dangerous comparisons:
- rewardsVaultAvailableBalance < currentAccruedRewards (src/CalderaStaking.sol#288)
- block.timestamp ≥ rewardsEndTimeStamp (src/CalderaStaking.sol#292)
CalderaStaking.stake(uint256) (src/CalderaStaking.sol#322-346) uses timestamp for comparisons
Dangerous comparisons:
- lastDepositedTimestamp ≠ 0 && lastDepositedTimestamp + 60 > block.timestamp (src/CalderaStaking.sol#331)
CalderaStaking.unstake(uint256) (src/CalderaStaking.sol#381-424) uses timestamp for comparisons
Dangerous comparisons:
- lastRequestedTimestamp ≠ 0 && lastRequestedTimestamp + 60 > block.timestamp (src/CalderaStaking.sol#393)
CalderaStaking.withdraw(uint256) (src/CalderaStaking.sol#428-462) uses timestamp for comparisons
Dangerous comparisons:
- block.timestamp < withdrawalRequest_.cooldownPeriodEndTimeStamp (src/CalderaStaking.sol#441)
CalderaStaking.getReward() (src/CalderaStaking.sol#466-486) uses timestamp for comparisons
Dangerous comparisons:
- claimableRewardAmount == 0 (src/CalderaStaking.sol#478)
- rewardsVaultBalance < claimableRewardAmount (src/CalderaStaking.sol#481)
CalderaStaking._cleanUserStakes(address) (src/CalderaStaking.sol#598-632) uses timestamp for comparisons
Dangerous comparisons:
- cleanedStakesReward > 0 (src/CalderaStaking.sol#617)
CalderaStaking._getUserWithdrawableAmount(address) (src/CalderaStaking.sol#683-691) uses timestamp for comparisons
Dangerous comparisons:
- block.timestamp ≥ userWithdrawalRequests[i].cooldownPeriodEndTimeStamp (src/CalderaStaking.sol#688)
CalderaStaking._isStakeLocked(CalderaStaking.Stake, uint256) (src/CalderaStaking.sol#697-699) uses timestamp for comparisons
Dangerous comparisons:
- stake_.lockedUntilTimestamp > atTimestamp (src/CalderaStaking.sol#698)
Reference: https://github.com/crytic/sliether/wiki/Detector-Documentation#block-timestamp
INFO:Sliether: analyzed (40 contracts with 100 detectors), 23 result(s) found

```

6. RISK METHODOLOGY

Every vulnerability and issue observed by Halborn is ranked based on **two sets of Metrics** and a **Severity Coefficient**. This system is inspired by the industry standard Common Vulnerability Scoring System.

The two **Metric sets** are: **Exploitability** and **Impact**. **Exploitability** captures the ease and technical means by which vulnerabilities can be exploited and **Impact** describes the consequences of a successful exploit.

The **Severity Coefficients** is designed to further refine the accuracy of the ranking with two factors: **Reversibility** and **Scope**. These capture the impact of the vulnerability on the environment as well as the number of users and smart contracts affected.

The final score is a value between 0-10 rounded up to 1 decimal place and 10 corresponding to the highest security risk. This provides an objective and accurate rating of the severity of security vulnerabilities in smart contracts.

The system is designed to assist in identifying and prioritizing vulnerabilities based on their level of risk to address the most critical issues in a timely manner.

6.1 EXPLOITABILITY

ATTACK ORIGIN (AO):

Captures whether the attack requires compromising a specific account.

ATTACK COST (AC):

Captures the cost of exploiting the vulnerability incurred by the attacker relative to sending a single transaction on the relevant blockchain. Includes but is not limited to financial and computational cost.

ATTACK COMPLEXITY (AX):

Describes the conditions beyond the attacker's control that must exist in order to exploit the vulnerability. Includes but is not limited to macro situation, available third-party liquidity and regulatory challenges.

METRICS:

EXPLOITABILITY METRIC (M_E)	METRIC VALUE	NUMERICAL VALUE
Attack Origin (AO)	Arbitrary (AO:A) Specific (AO:S)	1 0.2
Attack Cost (AC)	Low (AC:L) Medium (AC:M) High (AC:H)	1 0.67 0.33
Attack Complexity (AX)	Low (AX:L) Medium (AX:M) High (AX:H)	1 0.67 0.33

Exploitability E is calculated using the following formula:

$$E = \prod m_e$$

6.2 IMPACT

CONFIDENTIALITY (C):

Measures the impact to the confidentiality of the information resources managed by the contract due to a successfully exploited vulnerability. Confidentiality refers to limiting access to authorized users only.

INTEGRITY (I):

Measures the impact to integrity of a successfully exploited vulnerability. Integrity refers to the trustworthiness and veracity of data stored and/or processed on-chain. Integrity impact directly affecting Deposit or Yield records is excluded.

AVAILABILITY (A):

Measures the impact to the availability of the impacted component resulting from a successfully exploited vulnerability. This metric refers to smart contract features and functionality, not state. Availability impact directly affecting Deposit or Yield is excluded.

DEPOSIT (D):

Measures the impact to the deposits made to the contract by either users or owners.

YIELD (Y):

Measures the impact to the yield generated by the contract for either users or owners.

METRICS:

IMPACT METRIC (M_I)	METRIC VALUE	NUMERICAL VALUE
Confidentiality (C)	None (I:N) Low (I:L) Medium (I:M) High (I:H) Critical (I:C)	0 0.25 0.5 0.75 1
Integrity (I)	None (I:N) Low (I:L) Medium (I:M) High (I:H) Critical (I:C)	0 0.25 0.5 0.75 1
Availability (A)	None (A:N) Low (A:L) Medium (A:M) High (A:H) Critical (A:C)	0 0.25 0.5 0.75 1
Deposit (D)	None (D:N) Low (D:L) Medium (D:M) High (D:H) Critical (D:C)	0 0.25 0.5 0.75 1
Yield (Y)	None (Y:N) Low (Y:L) Medium (Y:M) High (Y:H) Critical (Y:C)	0 0.25 0.5 0.75 1

Impact I is calculated using the following formula:

$$I = \max(m_I) + \frac{\sum m_I - \max(m_I)}{4}$$

6.3 SEVERITY COEFFICIENT

REVERSIBILITY (R):

Describes the share of the exploited vulnerability effects that can be reversed. For upgradeable contracts, assume the contract private key is available.

SCOPE (S):

Captures whether a vulnerability in one vulnerable contract impacts resources in other contracts.

METRICS:

SEVERITY COEFFICIENT (<i>C</i>)	COEFFICIENT VALUE	NUMERICAL VALUE
Reversibility (<i>r</i>)	None (R:N) Partial (R:P) Full (R:F)	1 0.5 0.25
Scope (<i>s</i>)	Changed (S:C) Unchanged (S:U)	1.25 1

Severity Coefficient *C* is obtained by the following product:

$$C = rs$$

The Vulnerability Severity Score *S* is obtained by:

$$S = min(10, EIC * 10)$$

The score is rounded up to 1 decimal places.

SEVERITY	SCORE VALUE RANGE
Critical	9 - 10
High	7 - 8.9
Medium	4.5 - 6.9
Low	2 - 4.4

SEVERITY	SCORE VALUE RANGE
Informational	0 - 1.9

7. SCOPE

FILES AND REPOSITORY
(a) Repository: rollup-token-contracts-halborn (b) Assessed Commit ID: 37665f8 (c) Items in scope: - src/CalderaStaking.sol - src/CalderaAirdrop.sol - nate-caldera/era-contracts-halborn/pull/1/files
Out-of-Scope: Third party dependencies and economic attacks.
REMEDIATION COMMIT ID:
33a0850 - ec5f1ea - fc69e7c - acbca36 62f437a 7b4dd29 05c47ce - c7303f1 - c2e9028 0e6d31a 12e8cce
Out-of-Scope: New features/implementations after the remediation commit IDs.

8. ASSESSMENT SUMMARY & FINDINGS OVERVIEW

CRITICAL	HIGH	MEDIUM	LOW
0	0	2	2

INFORMATIONAL 12

SECURITY ANALYSIS	RISK LEVEL	REMEDIATION DATE
HAL-01 - DOS VULNERABILITY WHEN MINIMUM STAKE AMOUNT IS INCREASED	MEDIUM	SOLVED - 03/24/2025
HAL-02 - REWARD RATE NOT UPDATED AFTER CHANGE IN REWARDS END TIMESTAMP	MEDIUM	SOLVED - 03/24/2025
HAL-03 - INCONSISTENT UPGRADEABLE INHERITANCE PATTERN	LOW	SOLVED - 03/24/2025
HAL-04 - REWARD RATE SPIKE WHEN APPROACHING REWARD PERIOD END	LOW	SOLVED - 03/24/2025
HAL-05 - CENTRALIZATION RISK IN MANUAL REWARD RATE CONTROLS	INFORMATIONAL	SOLVED - 03/24/2025
HAL-06 - MISSING SETTER FUNCTION FOR AUTOMATICSTAKINGPERCENT	INFORMATIONAL	ACKNOWLEDGED - 03/24/2025
HAL-07 - INCONSISTENT INPUT VALIDATION ACROSS CONTRACT PARAMETERS	INFORMATIONAL	SOLVED - 03/24/2025
HAL-08 - ORDERING FOR NONREENTRANT MODIFIER	INFORMATIONAL	SOLVED - 03/24/2025

SECURITY ANALYSIS	RISK LEVEL	REMEDiation DATE
HAL-09 - LACK OF UPGRADEABLE STORAGE MANAGEMENT PATTERN	INFORMATIONAL	ACKNOWLEDGED - 03/21/2025
HAL-10 - USE OF MAGIC NUMBERS	INFORMATIONAL	SOLVED - 03/24/2025
HAL-11 - LACK OF NAMED MAPPINGS	INFORMATIONAL	SOLVED - 03/24/2025
HAL-12 - UNOPTIMIZED FOR LOOPS	INFORMATIONAL	SOLVED - 03/25/2025
HAL-13 - SINGLE STEP OWNERSHIP TRANSFER PROCESS	INFORMATIONAL	PARTIALLY SOLVED - 03/24/2025
HAL-14 - REDUNDANT/INEFFICIENT CODE	INFORMATIONAL	SOLVED - 03/24/2025
HAL-15 - INCONSISTENT FUNCTION NAMING CONVENTION	INFORMATIONAL	SOLVED - 03/24/2025
HAL-16 - INTEGER DIVISION IN REWARD RATE CALCULATION COULD LEAD TO ZERO REWARDS	INFORMATIONAL	NOT APPLICABLE - 03/21/2025

9. FINDINGS & TECH DETAILS

9.1 (HAL-01) DOS VULNERABILITY WHEN MINIMUM STAKE AMOUNT IS INCREASED

// MEDIUM

Description

The contract allows the owner to increase the minimum stake amount through the `setMinimumStakeAmount` function, which can create a denial-of-service condition for users who have already staked or unstaked amounts below the new minimum threshold. Both the `unstake()` and `withdraw()` functions enforce this minimum amount checks:

- In `unstake()`:

```
if (amount < minimumStakeAmount) {
    revert UnstakeAmountTooSmall();
}
```

- In `withdraw()`:

```
if (amount < minimumStakeAmount) {
    revert WithdrawAmountTooSmall();
}
```

For example, if Alice stakes `1e18` tokens when the `minimumStakeAmount` is `1e18`, and then the owner increases `minimumStakeAmount` to `2e18`:

1. Alice cannot unstake her `1e18` tokens because the transaction would revert with `UnstakeAmountTooSmall()`.
2. If Alice had already unstaked before the increase but the cooldown period hasn't ended, she would be unable to withdraw her `1e18` tokens after the cooldown because the transaction would revert with `WithdrawAmountTooSmall()`.

This effectively locks user funds until either:

- The owner decreases the minimum stake amount.
- The user acquires more tokens to meet the new minimum requirement.

BVSS

Recommendation

Consider one of these approaches:

1. Track the minimum stake amount for each stake/withdrawal request at the time it was created.
2. Add additional functionality that allows to "unstake all" and "withdraw all" function that bypasses the minimum amount check.
3. Keep the minimum amount check for new stakes, but remove it from unstaking and withdrawals, as users should always be able to withdraw funds they've already deposited.

Remediation Comment

SOLVED: The **Caldera team** solved this finding in commit **33a0850** by following the mentioned recommendation and adding additional functionality that allows to "unstake all" and "withdraw all".

Remediation Hash

<https://github.com/nate-caldera/era-contracts-halborn/commit/33a085007c547996d91aa3d1057a1b3b1c5cab86>

References

[nate-caldera/era-contracts-halborn/src/CalderaStaking.sol#L470-L472](https://github.com/nate-caldera/era-contracts-halborn/src/CalderaStaking.sol#L470-L472)

9.2 (HAL-02) REWARD RATE NOT UPDATED AFTER CHANGE IN REWARDS END TIMESTAMP

// MEDIUM

Description

When changing the rewards end timestamp by calling `setRewardsEndTimestamp()`, the contract doesn't automatically update the `rewardRate` to match the new distribution timeframe. This oversight leads to two problematic scenarios:

1. If extending the rewards period: The contract continues distributing at the original rate over a longer timeframe, significantly increasing the total rewards distributed beyond what was intended. For example, doubling the distribution period without adjusting the rate would distribute twice the planned rewards, potentially depleting the rewards vault prematurely.
2. If shortening the rewards period: The contract maintains the original rate but for a shorter duration, resulting in undistributed rewards that remain locked in the vault and unavailable to stakers.

The `rewardRate` is calculated in the `syncRewardRate()` function using the formula:

```
rewardRate = availableRewards / (rewardsEndTimestamp - blockTimestamp);
```

Since this calculation directly depends on `rewardsEndTimestamp`, any change to this parameter fundamentally alters the distribution mathematics, requiring a corresponding update to the reward rate.

Proof of Concept

In the following simplified PoC, the issue is demonstrated:

```
function test_setRewardsEndTimestampIssue() public {
    // Setup - use simple numbers for clarity
    uint256 initialRewardAmount = 1000 ether;
    deal(address(token), rewardsVault, initialRewardAmount);
    vm.prank(rewardsVault);
    token.approve(address(staking), type(uint256).max);

    // Initial setup: 10-day rewards period
    uint256 initialPeriod = 10 days;
    vm.prank(owner);
    staking.setRewardsEndTimestamp(block.timestamp + initialPeriod);

    // Set initial reward rate to distribute 500 tokens over 10 days
    uint256 targetRate = 500 ether / initialPeriod; // 50 tokens per day
    vm.prank(owner);
    staking.dangerouslySetRewardRate(targetRate, true);

    // User stakes
    vm.startPrank(user1);
    token.approve(address(staking), STAKE_AMOUNT);
    staking.stake(STAKE_AMOUNT);
    vm.stopPrank();

    console.log("----- SCENARIO: EXTENDING THE REWARDS PERIOD -----");
    console.log("Initial setup: 500 tokens to be distributed over 10 days");
```

```

console.log("Daily distribution rate: 50 tokens per day");

// Fast forward 5 days into the staking period
vm.warp(block.timestamp + 5 days);

console.log("After 5 days:");
console.log("- Rewards earned: %d tokens", staking.getUserTotalRewardAmount(user1) / 1 ether);
console.log("- Rewards remaining to distribute: %d tokens", 500 - staking.getUserTotalRewardAmount(user1) / 1 ether);

// SCENARIO: Extend the rewards period by 10 more days (to 20 days total)
vm.prank(owner);
staking.setRewardsEndTimestamp(block.timestamp + 15 days); // +15 from current time = 20 days total

console.log("\nOWNER ACTION: Extended rewards period from 10 days to 20 days total");
console.log("BUT reward rate not adjusted - still distributing 50 tokens per day");

// Fast forward to end of original period (5 more days)
vm.warp(block.timestamp + 5 days);

console.log("\nAt original end date (10 days total):");
console.log("- Rewards earned: %d tokens", staking.getUserTotalRewardAmount(user1) / 1 ether);

// Fast forward to end of extended period (10 more days)
vm.warp(block.timestamp + 10 days);

console.log("\nAt extended end date (20 days total):");
console.log("- Rewards earned: %d tokens", staking.getUserTotalRewardAmount(user1) / 1 ether);
console.log("- Original intended distribution: 500 tokens");
console.log("- Excess distributed: %d tokens", staking.getUserTotalRewardAmount(user1) / 1 ether - 500);

// Claim rewards to see the actual transfer
uint256 userBalanceBefore = token.balanceOf(user1) / 1 ether;
uint256 vaultBalanceBefore = token.balanceOf(rewardsVault) / 1 ether;

vm.warp(block.timestamp + staking.stakeLockupPeriod() + 1);

vm.prank(user1);
staking.getReward();

uint256 userBalanceAfter = token.balanceOf(user1) / 1 ether;
uint256 vaultBalanceAfter = token.balanceOf(rewardsVault) / 1 ether;

console.log("\nBalances after claiming:");
console.log("- User received: %d tokens", userBalanceAfter - userBalanceBefore);
console.log("- Vault paid out: %d tokens", vaultBalanceBefore - vaultBalanceAfter);

console.log("\nCORRECT BEHAVIOR WOULD BE:");
console.log("1. When extending period from 10 to 20 days with 250 tokens remaining");
console.log("2. Reward rate should adjust from 50 tokens/day to 25 tokens/day");
console.log("3. Total distribution should remain 500 tokens (not 1000)");
}

```

Ran 1 test for test/CalderaStaking.t.sol:CalderaStakingTest

[PASS] test_setRewardsEndTimestampIssue() (gas: 519873)

Logs:

————— SCENARIO: EXTENDING THE REWARDS PERIOD —————

Initial setup: 500 tokens to be distributed over 10 days

Daily distribution rate: 50 tokens per day

After 5 days:

- Rewards earned: 249 tokens
- Rewards remaining to distribute: 251 tokens

OWNER ACTION: Extended rewards period from 10 days to 20 days total

BUT reward rate not adjusted - still distributing 50 tokens per day

At original end date (10 days total):

- Rewards earned: 499 tokens

At extended end date (20 days total):

- Rewards earned: 999 tokens
- Original intended distribution: 500 tokens
- Excess distributed: 499 tokens

Balances after claiming:

- User received: 999 tokens
- Vault paid out: 1000 tokens

CORRECT BEHAVIOR WOULD BE:

1. When extending period from 10 to 20 days with 250 tokens remaining
2. Reward rate should adjust from 50 tokens/day to 25 tokens/day
3. Total distribution should remain 500 tokens (not 1000)

Suite result: **ok**. 1 passed; 0 failed; 0 skipped; finished in 1.73ms (510.74µs CPU time)

BVSS

AO:A/AC:L/AX:L/R:N/S:U/C:N/A:N/I:M/D:N/Y:N (5.0)

Recommendation

Enforce a call to the **syncRewardRate()** function after calling **setRewardsEndTimestamp()** to ensure the reward rate aligns with the new distribution period to prevent unintended economic consequences and ensure that rewards are distributed according to the protocol's intended schedule, regardless of timestamp modifications.

Remediation Comment

SOLVED: The **Caldera team** solved this finding in commit **ec5f1ea** by following the mentioned recommendation and adding an additional parameter **shouldSyncRewardRate** that gives the contract /

owner flexibility to decide whether to adjust the reward rate when changing the end timestamp.

Remediation Hash

<https://github.com/nate-caldera/era-contracts-halborn/commit/ec5f1ea0faec02ec62f88329fa72d49791843a48>

References

[nate-caldera/era-contracts-halborn/src/CalderaStaking.sol#L298-L322](#)

[nate-caldera/era-contracts-halborn/src/CalderaStaking.sol#L338-L366](#)

9.3 (HAL-03) INCONSISTENT UPGRADEABLE INHERITANCE PATTERN

// LOW

Description

The `CalderaStaking` contract inherits from several upgradeable contracts (`UUPSUpgradeable`, `PausableUpgradeable`, and `OwnableUpgradeable`) but also inherits from the non-upgradeable `ReentrancyGuard`. This inconsistent inheritance pattern could potentially lead to issues during contract upgrades, since the non-upgradeable `ReentrancyGuard` initializes its state variables in the constructor rather than in an initializer function.

BVSS

AO:S/AC:L/AX:L/R:N/S:U/C:N/A:C/I:N/D:N/Y:N (2.0)

Recommendation

Replace the import and inheritance of `ReentrancyGuard` with `ReentrancyGuardUpgradeable` from the OpenZeppelin upgradeable contracts library, and add its initializer call in the `initialize` function.

Remediation Comment

SOLVED: The **Caldera team** solved this finding in commit `fc69e7c` by following the mentioned recommendation.

Remediation Hash

<https://github.com/nate-caldera/era-contracts-halborn/commit/fc69e7cf8d83b254cdb85d60132897950cffd755>

References

[nate-caldera/era-contracts-halborn/src/CalderaStaking.sol#L18](https://github.com/nate-caldera/era-contracts-halborn/blob/main/src/CalderaStaking.sol#L18)

9.4 (HAL-04) REWARD RATE SPIKE WHEN APPROACHING REWARD PERIOD END

// LOW

Description

In the `syncRewardRate` function, a significant vulnerability may arise when calculating the `rewardRate` near the end of the rewards period.

When `rewardsEndTimestamp` is very close to `blockTimestamp`, the division operation `availableRewards / (rewardsEndTimestamp - blockTimestamp)` can produce an extremely high reward rate due to the small denominator:

```
function syncRewardRate(uint256 rewardsVaultTargetPercent)
    external
    onlyOwner
    nonReentrant
    updateReward(address(0))
{
    if (rewardsVaultTargetPercent > 1e18) {
        revert InvalidRewardsVaultTargetPercent();
    }

    uint256 rewardsVaultAvailableBalance = IERC20(token).balanceOf(rewardsVault) * rewardsVaultTargetPercent / 1e18;
    uint256 currentAccruedRewards = accruedRewards + _getRewardsAccruedSinceLastUpdate();

    if (rewardsVaultAvailableBalance < currentAccruedRewards) {
        revert InsufficientRewardsVaultBalance();
    }
    uint256 previousRewardRate = rewardRate;
    uint256 blockTimestamp = block.timestamp;

    if (blockTimestamp >= rewardsEndTimestamp) {
        rewardRate = 0;
        emit RewardRateSet(previousRewardRate, rewardRate);
        return;
    }

    uint256 availableRewards = rewardsVaultAvailableBalance - currentAccruedRewards;
    rewardRate = availableRewards / (rewardsEndTimestamp - blockTimestamp);
    emit RewardRateSet(previousRewardRate, rewardRate);
}
```

This creates an issue where users who interact with the contract in the final seconds can receive disproportionately large rewards. This effectively allows the user who first calls `getReward()` after the rate spike to drain a significant portion of the remaining rewards in seconds.

While only the owner can trigger this condition by calling `syncRewardRate` near the end of the rewards period, it represents an economic vulnerability that could occur accidentally and significantly disrupt the intended tokenomics of the protocol.

Proof of Concept

The following test demonstrates that when only 2 seconds remain in the rewards period and `syncRewardRate` is called, the `rewardRate` spikes dramatically:

```
function test_rewardRateCalculationVulnerability() public {
    // Setup - add tokens to rewards vault
    uint256 initialRewardAmount = 1_000_000 ether;
    deal(address(token), rewardsVault, initialRewardAmount * 20);

    vm.prank(rewardsVault);
    token.approve(address(staking), type(uint256).max);

    // Set a 30-day reward period
    vm.prank(owner);
    staking.setRewardsEndTimestamp(block.timestamp + 30 days);

    // User stakes tokens
    vm.startPrank(user1);
    token.approve(address(staking), STAKE_AMOUNT);
    staking.stake(STAKE_AMOUNT);
    vm.stopPrank();

    // Set a low initial reward rate so rewards don't accrue quickly
    vm.prank(owner);
    staking.dangerouslySetRewardRate(1 ether, true); // Just 1 token per second

    // Fast forward to just before the end of the rewards period (2 seconds remaining)
    vm.warp(staking.rewardsEndTimestamp() - 2);

    // Log the state before syncing reward rate
    console.log("Accrued rewards before resync:", staking.accruedRewards());
    console.log("Vault balance:", token.balanceOf(rewardsVault));

    // Now sync the reward rate when only 2 seconds remain in the period
    vm.prank(owner);
    staking.syncRewardRate(1e18); // Use 100% of available rewards

    // Check the new reward rate
    uint256 spikedRate = staking.rewardRate();
    console.log("New reward rate:", spikedRate);
    console.log("Tokens per second:", spikedRate / 1e18);

    // Fast forward 1 second to accrue rewards at the extreme rate
    vm.warp(block.timestamp + 1);

    // Check how many rewards were earned in just 1 second
    uint256 earnedInOneSecond = staking.earned(getStake(user1, 0));
    console.log("Rewards earned in 1 second:", earnedInOneSecond / 1e18);

    // Calculate what percentage of total rewards this represents
    uint256 percentOfTotal = (earnedInOneSecond * 100) / initialRewardAmount;
    console.log("Percent of total rewards earned in 1 second:", percentOfTotal, "%");

    // Assert that a significant percentage of rewards can be earned in 1 second
    assertGt(percentOfTotal, 25, "User should earn >25% of total rewards in 1 second");
}
```

```
Ran 1 test for test/CalderaStaking.t.sol:CalderaStakingTest
[PASS] test_rewardRateCalculationVulnerability2() (gas: 473195)
```

Logs:

Accrued rewards before resync: 0

Vault balance: 2000000000000000000000000000

New reward rate: 870400100000000000000000

Tokens per second: 8704001

amount 1000000000000000000000000000

```
depositedTimestamp 1
```

```
lockedUntilTimestamp 7776001
```

```
rewardPerTokenPaid 0
```

reward 0

Rewards earned in 1 second: 11295999

Percent of total rewards earned in 1 second: 1129 %

Suite result: **ok**. 1 passed; 0 failed; 0 skipped; finished in 1.87ms (468.01µs CPU time)

BVSS

AO:S/AC:L/AX:L/R:N/S:U/C:N/A:N/I:N/D:N/Y:C (2.0)

Recommendation

To prevent reward rate manipulation near the end of a reward period, consider implementing one or more safeguards in the `syncRewardRate()` function:

- **Minimum Time Threshold:** Require a minimum remaining duration before allowing `syncRewardRate()` to be called, preventing last-minute manipulation.
- **Maximum Reward Rate Cap:** Set an upper boundary on the reward rate based on protocol economics to prevent extreme values.

These measures help ensure stability and mitigate the risk of excessively high reward rates.

Remediation Comment

SOLVED: The **Caldera team** solved this finding in commit **acbca36** by following the mentioned recommendation.

Remediation Hash

<https://github.com/nate-caldera/era-contracts-halborn/commit/acbca36f69d19b3130ddc416055c688a3ef2ea59>

References

[nate-caldera/era-contracts-halborn/src/CalderaStaking.sol#L364](https://github.com/nate-caldera/era-contracts-halborn/src/CalderaStaking.sol#L364)

9.5 (HAL-05) CENTRALIZATION RISK IN MANUAL REWARD RATE CONTROLS

// INFORMATIONAL

Description

The contract includes a `dangerouslySetRewardRate()` function that allows the owner to directly set the reward rate, bypassing the safety checks and calculations present in the recommended `syncRewardRate()` function. While this function requires confirmation through a boolean parameter and includes a descriptive warning, it lacks any mathematical constraints or validation on the reward rate value itself. This override capability could serve legitimate purposes:

- Emergency intervention when automatic calculations produce undesired results.
- Adjusting rewards during exceptional market conditions.
- Handling edge cases not accounted for in the automatic calculation.

However, the function introduces significant economic and security concerns, since an excessively high reward rate could rapidly deplete the rewards vault, unfairly benefiting users who claim rewards first.

BVSS

AO:S/AC:L/AX:L/R:N/S:U/C:N/A:N/I:N/D:H/Y:H (1.9)

Recommendation

Consider one or more of the following:

1. Implement upper and lower bound validations for the reward rate value relative to available rewards.
2. Add a timelock mechanism, requiring a delay before manual rate changes take effect.
3. Consider implementing governance approval for manual rate changes.
4. If retained as an emergency function, add additional safeguards like maximum deviation from calculated rate.

While emergency controls can be valuable, this function represents a significant centralization risk that could undermine the protocol's economic stability and user trust.

Remediation Comment

SOLVED: The **Caldera team** solved this finding in commit [acbca36](#) by following the mentioned recommendation and implementing an upper validation for the new reward rate.

Remediation Hash

<https://github.com/nate-caldera/era-contracts-halborn/commit/acbca36f69d19b3130ddc416055c688a3ef2ea59>

References

[nate-caldera/era-contracts-halborn/src/CalderaStaking.sol#L378](#)

9.6 (HAL-06) MISSING SETTER FUNCTION FOR AUTOMATICSTAKINGPERCENT

// INFORMATIONAL

Description

The **CalderaAirdrop** contract has a public variable **automaticStakingPercent** that is set during initialization but lacks a setter function. This means that the automatic staking percentage cannot be adjusted after deployment, limiting the contract's flexibility.

BVSS

[AO:A/AC:H/AX:L/R:N/S:U/C:N/A:N/I:L/D:N/Y:N](#) (0.8)

Recommendation

Add a public setter function for **automaticStakingPercent** to allow the contract owner to adjust the automatic staking percentage as needed.

Remediation Comment

ACKNOWLEDGED: The **Caldera team** made a business decision to acknowledge this finding and not alter the contracts.

References

[nate-caldera/era-contracts-halborn/src/CalderaAirdrop.sol#L50](#)

9.7 (HAL-07) INCONSISTENT INPUT VALIDATION ACROSS CONTRACT PARAMETERS

// INFORMATIONAL

Description

The contract implements validation checks inconsistently across different parameter-setting functions. While some parameters have robust validation including both lower and upper bounds, others have only partial checks or lack important validations altogether.

Instances of this issue include:

1. In the `initialize` function, `rewardsEndTimestamp_` is only checked to be non-zero, but unlike the `setRewardsEndTimestamp` function, it doesn't verify that the timestamp is in the future. If initialized with a past timestamp, rewards would effectively be disabled from the start, as `lastTimeRewardApplicable()` would return the past timestamp rather than the current time.
2. The `setMinimumStakeAmount()` function only verifies that the new minimum stake amount is non-zero but lacks an upper bound check. Setting an excessively high minimum stake amount could prevent users from staking, unstaking or withdrawing altogether if their amounts don't meet the new threshold.

These inconsistent validation patterns could lead to contract misconfiguration that impacts user operations or rewards distribution.

BVSS

AO:S/AC:L/AX:L/R:N/S:U/C:N/A:L/I:N/D:N/Y:N (0.5)

Recommendation

Apply consistent validation logic across all parameter-setting functions:

1. Add a check in the `initialize()` function to ensure that `rewardsEndTimestamp_` is in the future, similar to the check in `setRewardsEndTimestamp`.
2. For `setMinimumStakeAmount`, add an upper bound check to prevent setting unreasonably high minimum stake amounts.

Remediation Comment

SOLVED: The **Caldera team** solved this finding in commit **62f437a** by following the mentioned recommendations.

Remediation Hash

<https://github.com/nate-caldera/era-contracts-halborn/commit/62f437a46911cb951dd83b91b64a9ca9c6ec3f75>

References

[nate-caldera/era-contracts-halborn/src/CalderaStaking.sol#L154-L211](#)

[nate-caldera/era-contracts-halborn/src/CalderaStaking.sol#L266-L274](#)

9.8 (HAL-08) ORDERING FOR NONREENTRANT MODIFIER

// INFORMATIONAL

Description

In Solidity, if a function has multiple modifiers, they are executed in the order specified. If checks or logic of modifiers depend on other modifiers, this has to be considered in their ordering.

Several functions of the contracts in scope have multiple modifiers, with one of them being **nonReentrant** which prevents reentrancy behavior on the functions. Ideally, the **nonReentrant** modifier should be the first one to prevent even the execution of other modifiers in case of reentrancy behavior.

While there is currently no obvious vulnerability with **nonReentrant** not being the first modifier, placing it first ensures that all other modifiers are executed only if the call is non-reentrant. This is a safer practice and can prevent potential issues in future updates or unforeseen scenarios.

BVSS

AO:A/AC:H/AX:H/R:N/S:U/C:N/A:N/I:N/D:L/Y:N (0.3)

Recommendation

Switch modifier order to consistently place the **nonReentrant** modifier as the first one to run so that all other modifiers are executed only if the call is non-reentrant.

Remediation Comment

SOLVED: The **Caldera team** solved this finding in commit **7b4dd29** by following the mentioned recommendation.

Remediation Hash

<https://github.com/nate-caldera/era-contracts-halborn/commit/7b4dd2991750785bfa5b2cb138e682d2c892194d>

References

[nate-caldera/era-contracts-halborn/src/CalderaStaking.sol#L341](#)

[nate-caldera/era-contracts-halborn/src/CalderaStaking.sol#L392](#)

[nate-caldera/era-contracts-halborn/src/CalderaStaking.sol#L432](#)

[nate-caldera/era-contracts-halborn/src/CalderaStaking.sol#L469](#)

[nate-caldera/era-contracts-halborn/src/CalderaStaking.sol#L534](#)

[nate-caldera/era-contracts-halborn/src/CalderaStaking.sol#L578](#)

9.9 (HAL-09) LACK OF UPGRADEABLE STORAGE MANAGEMENT PATTERN

// INFORMATIONAL

Description

The contracts in scope use UUPS upgradeable pattern but it is currently missing storage gaps or namespaced storage pattern to manage its storage layout.

The usage of a storage gap was encouraged in OpenZeppelin Upgradeable Contracts up to version 4, where storage gaps (like the `uint256[50] private __gap;` array) are used to reserve storage slots for future variables, helping to prevent storage collisions during upgrades.

However, starting from OpenZeppelin Contracts version 5, the recommended best practice has shifted towards using Namespaced storage. Namespaced storage provides better isolation of storage variables, significantly reducing the risk of storage collisions, especially in complex inheritance structures common with upgradeable contracts.

BVSS

AO:S/AC:M/AX:M/R:N/S:U/C:N/A:N/I:L/D:N/Y:N (0.2)

Recommendation

Refactor the contracts to use namespaced storage patterns, as recommended in OpenZeppelin Contracts v5.

Encapsulate the contract's storage variables within a struct and associate it with a unique storage slot, ensuring isolation from inherited contracts' storage.

Remediation Comment

ACKNOWLEDGED: The **Caldera team** made a business decision to acknowledge this finding and not alter the contracts, stating that:

We will not inherit from these contracts and/or change the storage order when upgrading.

References

[nate-caldera/era-contracts-halborn/src/CalderaAirdrop.sol#L41](#)

[nate-caldera/era-contracts-halborn/src/CalderaStaking.sol#L18](#)

9.10 (HAL-10) USE OF MAGIC NUMBERS

// INFORMATIONAL

Description

Throughout the contracts in scope, there are several instances where magic numbers are used. Magic numbers are direct numerical or string values used in the code without any explanation or context. This practice can lead to code maintainability issues, potential errors, and difficulty in understanding the code's logic.

BVSS

AO:A/AC:L/AX:L/R:N/S:U/C:N/A:N/I:N/D:N/Y:N (0.0)

Recommendation

To improve the code's readability and facilitate refactoring, consider defining a constant for every magic number, giving it a clear and self-explanatory name. Consider adding an inline comment explaining how the magic numbers are calculated or why they are chosen for complex values.

Remediation Comment

SOLVED: The **Caldera team** solved this finding in commit **05c47ce** by adding inline comments explaining the values for the magic numbers.

Remediation Hash

<https://github.com/nate-caldera/era-contracts-halborn/commit/05c47cea9ba6b8f3f67fabbce3289b026ceb6df7>

References

[nate-caldera/era-contracts-halborn/src/CalderaStaking.sol#L344](#)
[nate-caldera/era-contracts-halborn/src/CalderaStaking.sol#L348](#)
[nate-caldera/era-contracts-halborn/src/CalderaStaking.sol#L733](#)
[nate-caldera/era-contracts-halborn/src/CalderaStaking.sol#L726](#)
[nate-caldera/era-contracts-halborn/src/CalderaStaking.sol#L624](#)
[nate-caldera/era-contracts-halborn/src/CalderaAirdrop.sol#L390](#)
[nate-caldera/era-contracts-halborn/src/CalderaAirdrop.sol#L417](#)

9.11 (HAL-11) LACK OF NAMED MAPPINGS

// INFORMATIONAL

Description

The project contains several unnamed mappings despite using a Solidity version that supports named mappings.

Named mappings improve code readability and self-documentation by explicitly stating their purpose.

BVSS

AO:A/AC:L/AX:L/R:N/S:U/C:N/A:N/I:N/D:N/Y:N (0.0)

Recommendation

Consider refactoring the mappings to use named arguments, which will enhance code readability and make the purpose of each mapping more explicit. For example:

```
mapping(address myAddress => bool myBool) public myMapping;
```

Remediation Comment

SOLVED: The **Caldera team** solved this finding in commit **c7303f1** by following the mentioned recommendation.

Remediation Hash

<https://github.com/nate-caldera/era-contracts-halborn/commit/c7303f1d174e32e64986a00e7f8ee814ce8dba57>

References

[nate-caldera/era-contracts-halborn/src/CalderaStaking.sol#L43-L44](https://github.com/nate-caldera/era-contracts-halborn/src/CalderaStaking.sol#L43-L44)

9.12 (HAL-12) UNOPTIMIZED FOR LOOPS

// INFORMATIONAL

Description

Throughout the code in scope, there are several instances of unoptimized for loop declarations that may incur in higher gas costs than necessary.

BVSS

AO:A/AC:L/AX:L/R:N/S:U/C:N/A:N/I:N/D:N/Y:N (0.0)

Recommendation

Optimize the **for** loop declarations to reduce gas costs. Best practices include:

- The non-redundant initialization of the iterator with a default value (declaring simply **i** is equivalent to **i = 0** but more gas efficient),
- The use of the pre-increment operator (inside an **unchecked** block if using Solidity **>=0.8.0** and **<=0.8.21** (**unchecked {++i}**), or simply **++i** if compiling with Solidity **>=0.8.22**).

Additionally,

- When reading from storage variables and if the iterations goes over the array multiple times, it is recommended to reduce gas costs significantly by caching the array to read locally and iterate over it to avoid reading from storage on every iteration,
- Avoid declaring new variables within each iteration; instead, declare them before the loop and update their values during each iteration.
- If there are several loops in the same function, the **i** variable can be re-used, to be able to set the value from non-zero to zero and reduce gas costs without additional variable declaration. For example:

```
uint256[] memory arrayInMemory = arrayInStorage;
Stake storage stake_;

uint256 i;

for (; i < arrayInMemory.length ; ++i) {
    stake_ = userStakes[i];
}

delete i;

uint256[] memory arrayInMemory2 = arrayInStorage2;

for (; i < arrayInMemory2.length ; ++i) {
    // code logic
}
```

Remediation Comment

SOLVED: The **Caldera team** solved this finding in commit **c2e9028** by following the mentioned recommendation.

Remediation Hash

<https://github.com/nate-caldera/era-contracts-halborn/commit/c2e9028b028108340fa413f65fc7826f5dfa5e62>

References

[nate-caldera/era-contracts-halborn/src/CalderaStaking.sol#L115](#)
[nate-caldera/era-contracts-halborn/src/CalderaStaking.sol#L498](#)
[nate-caldera/era-contracts-halborn/src/CalderaStaking.sol#L549](#)
[nate-caldera/era-contracts-halborn/src/CalderaStaking.sol#L583](#)
[nate-caldera/era-contracts-halborn/src/CalderaStaking.sol#L635](#)
[nate-caldera/era-contracts-halborn/src/CalderaStaking.sol#L655](#)
[nate-caldera/era-contracts-halborn/src/CalderaStaking.sol#L676](#)
[nate-caldera/era-contracts-halborn/src/CalderaStaking.sol#L696](#)
[nate-caldera/era-contracts-halborn/src/CalderaStaking.sol#L743](#)
[nate-caldera/era-contracts-halborn/src/CalderaStaking.sol#L784](#)
[nate-caldera/era-contracts-halborn/src/CalderaStaking.sol#L821](#)
[nate-caldera/era-contracts-halborn/src/CalderaStaking.sol#L836](#)

9.13 (HAL-13) SINGLE STEP OWNERSHIP TRANSFER PROCESS

// INFORMATIONAL

Description

The contracts in scope inherit the `OwnableUpgradeable` contract implementation from OpenZeppelin's library and is used to restrict access to certain functions to the contract owner. The `OwnableUpgradeable` pattern allows the contract owner to transfer ownership to another address using the `transferOwnership()` function. However, the `transferOwnership()` function does not include a two-step process to transfer ownership.

Regarding this, it is crucial that the address to which ownership is transferred is verified to be active and willing to assume ownership responsibilities. Otherwise, the contract could be locked in a situation where it is no longer possible to make administrative changes to it.

Additionally, the `renounceOwnership()` function allows renouncing to the owner permission. Renouncing ownership before transferring it would result in the contract having no owner, eliminating the ability to call privileged functions.

BVSS

AO:A/AC:L/AX:L/R:N/S:U/C:N/A:N/I:N/D:N/Y:N (0.0)

Recommendation

Consider using OpenZeppelin's `Ownable2StepUpgradeable` contract over the `OwnableUpgradeable` implementation. `Ownable2StepUpgradeable` provides a two-step ownership transfer process, which adds an extra layer of security to prevent accidental ownership transfers.

Additionally, it is recommended that the owner cannot call the `renounceOwnership()` function to avoid losing ownership of the contract.

Remediation Comment

PARTIALLY SOLVED: The **Caldera team** partially solved this finding in commits [0e6d31a](#) by following the mentioned recommendation and inherit from `Ownable2StepUpgradeable` but keeping the possibility to renounce ownership, stating:

We will retain the `renounceOwnership` function to give us the optionality to fully decentralize later.

Remediation Hash

<https://github.com/nate-caldera/era-contracts-halborn/commit/0e6d31af8f0334df560cc77a2c3045ecb9127d26>

References

[nate-caldera/era-contracts-halborn/src/CalderaStaking.sol#L18](#)

[nate-caldera/era-contracts-halborn/src/CalderaAirdrop.sol#L41](#)

9.14 (HAL-14) REDUNDANT/INEFFICIENT CODE

// INFORMATIONAL

Description

The CalderaStaking contract contains several instances of redundant or inefficient code patterns that could be optimized for gas efficiency and improved readability. Specific instances include:

1. Unnecessary explicit initialization of variables to zero, when Solidity already initializes variables to zero by default.
2. The function `getUserUnlockedStakeAmount()` is redundant as it simply calls the already public `_getUserUnlockedStakeAmount()` function without adding any functionality.
3. In the `setRewardsEndTimestamp()` function, the contract checks if the new rewards end timestamp is greater than 0 and also checks if it's greater than the current block timestamp. This is redundant as the second condition already implies the first.

```
if (rewardsEndTimestamp_ == 0 || rewardsEndTimestamp_ < block.timestamp) revert InvalidRewardsEndTimestamp();
```

BVSS

AO:A/AC:L/AX:L/R:N/S:U/C:N/A:N/I:N/D:N/Y:N (0.0)

Recommendation

1. Remove redundant zero initializations for variables.
2. In `setRewardsEndTimestamp()`, simplify the condition to only check if the timestamp is greater than the current block timestamp.
3. Remove the redundant `getUserUnlockedStakeAmount()` function since the internal function it calls is already public.

Remediation Comment

SOLVED: The Caldera team solved this finding in commits [12e8cce](#) and [62f437a](#) by following the mentioned recommendations.

Remediation Hash

<https://github.com/nate-caldera/era-contracts-halborn/commit/12e8cce51ee975557bca26bcfa7765a1a200cffb>

References

[nate-caldera/era-contracts-halborn/src/CalderaStaking.sol#L54](#)
[nate-caldera/era-contracts-halborn/src/CalderaStaking.sol#L632](#)
[nate-caldera/era-contracts-halborn/src/CalderaStaking.sol#L653](#)
[nate-caldera/era-contracts-halborn/src/CalderaStaking.sol#L673](#)
[nate-caldera/era-contracts-halborn/src/CalderaStaking.sol#L694](#)
[nate-caldera/era-contracts-halborn/src/CalderaStaking.sol#L740-L741](#)
[nate-caldera/era-contracts-halborn/src/CalderaStaking.sol#L782](#)
[nate-caldera/era-contracts-halborn/src/CalderaStaking.sol#L818](#)
[nate-caldera/era-contracts-halborn/src/CalderaStaking.sol#L834](#)

9.15 (HAL-15) INCONSISTENT FUNCTION NAMING CONVENTION

// INFORMATIONAL

Description

The contract uses inconsistent naming conventions for functions, particularly with two public functions prefixed with underscores: `_getUserUnlockedStakeAmount()` and `_getUserWithdrawableAmount()`. This contradicts Solidity's style guide recommendation that underscores should prefix internal and private function names, while public and external functions should not have underscore prefixes.

This inconsistency creates confusion about the intended visibility and usage of these functions. While these particular functions are correctly marked as `public`, the underscore prefix misleadingly suggests they are intended for internal use only.

BVSS

AO:A/AC:L/AX:L/R:N/S:U/C:N/A:N/I:N/D:N/Y:N (0.0)

Recommendation

Rename public functions to remove the underscore prefix, following Solidity's naming convention.

Remediation Comment

SOLVED: The **Caldera team** solved this finding in commit `12e8cce` by following the mentioned recommendation.

Remediation Hash

<https://github.com/nate-caldera/era-contracts-halborn/commit/12e8cce51ee975557bca26bcfa7765a1a200cffb>

References

[nate-caldera/era-contracts-halborn/src/CalderaStaking.sol#L816](#)

[nate-caldera/era-contracts-halborn/src/CalderaStaking.sol#L832](#)

9.16 (HAL-16) INTEGER DIVISION IN REWARD RATE CALCULATION COULD LEAD TO ZERO REWARDS

// INFORMATIONAL

Description

In the `syncRewardRate()` function, integer division when calculating the reward rate can lead to precision loss:

```
rewardRate = availableRewards / (rewardsEndTimestamp - blockTimestamp);
```

When `availableRewards` is small relative to the time period, this division rounds down to zero due to Solidity's integer arithmetic, resulting in no rewards being distributed despite tokens being available in the rewards vault.

This could lead to stakers receiving no rewards even when tokens are available, breaks the incentive mechanism of the protocol.

BVSS

AO:A/AC:L/AX:L/R:N/S:U/C:N/A:N/I:N/D:N/Y:N (0.0)

Recommendation

Add a validation check to ensure the reward rate is never unintentionally zero to guarantee that rewards will be distributed if they exist.

Remediation Comment

NOT APPLICABLE: After discussion with the **Caldera team**, it has been concluded that this issue is practically impossible to occur in real-life scenarios.

References

[nate-caldera/era-contracts-halborn/src/CalderaStaking.sol#L364](https://github.com/nate-caldera/era-contracts-halborn/src/CalderaStaking.sol#L364)

Halborn strongly recommends conducting a follow-up assessment of the project either within six months or immediately following any material changes to the codebase, whichever comes first. This approach is crucial for maintaining the project's integrity and addressing potential vulnerabilities introduced by code modifications.